

Table of Contents

Week 1 Summary — What is OOP?

Easy Explanation with Real-World Examples

Part 1 — What is a Programming Paradigm?

Simple Definition

Real-World Example: Building a House

Procedural Thinking

OOP Thinking

Key Message for Students

Part 2 — Problems with Procedural Programming

Simple Definition

Real-World Example: Bank Account

Data

Actions

Problem 1: Data Can Be Changed by Anyone

Problem 2: Too Many Variables

Problem 3: Hard to Manage Large Systems

School Analogy

Key Message for Students

Part 3 — How OOP Was Born

Why Was OOP Created?

Real-World Example: University System

OOP's Big Idea

Example: Student

Properties

Actions

Example: Car

Properties

Actions

Key Message for Students

Part 4 — The Four Pillars of OOP

1. Encapsulation

Real-World Example: ATM Machine

Student Summary

2. Inheritance

Meaning

Real-World Example: Person → Student

Person Has

Student Adds

Student Summary

3. Polymorphism

Meaning

Real-World Example: Animal Speaking

Dog

Cat

Cow

Student Summary

4. Abstraction

Meaning

Real-World Example: Driving a Car

Student Summary

Part 5 — First Java Class

What is a Class?

Real-World Example: House Blueprint

What is an Object?

Example

House Example

Student Object Example

Student Summary

Final Recap

What Students Must Remember

Programming Paradigm

Procedural Programming

OOP

The Four Pillars

Class vs Object

One-Sentence Definition of OOP

Week 1 Summary — What is OOP?

Easy Explanation with Real-World Examples

Course: Object-Oriented Programming Concepts

Week: 1

Duration: 1 Hour

Goal: Students understand what OOP is, why it exists, and the basic concepts of Class, Object, and the 4 Pillars of OOP.

Part 1 — What is a Programming Paradigm?

Simple Definition

A **Programming Paradigm** is a way of thinking about how to write a program.

Think of it as a strategy or style for solving problems with code.

Real-World Example: Building a House

Imagine you want to build a house.

Procedural Thinking

You focus on the steps:

1. Build foundation
2. Build walls
3. Install roof
4. Paint house

This is similar to following a recipe.

OOP Thinking

Instead of focusing on steps, you focus on the people involved:

- Architect
- Engineer
- Builder
- Electrician

Each person has:

- Their own information
- Their own responsibilities
- Their own tasks

This is how OOP works.

Key Message for Students

A Programming Paradigm is simply a style of organizing and solving problems using code.

Different paradigms answer the question:

“How should we organize our program?”

Part 2 — Problems with Procedural Programming

Simple Definition

Procedural Programming organizes programs using:

- Variables (Data)
- Functions (Actions)

The data and actions are stored separately.

Real-World Example: Bank Account

Imagine a bank account.

Data

- Owner = Sokha
- Balance = \$1000
- Type = Savings

Actions

- Deposit Money
- Withdraw Money
- Print Statement

In procedural programming, the data and actions are separated.

Problem 1: Data Can Be Changed by Anyone

Someone accidentally writes:

```
Balance = -999999
```

Now the account contains invalid data.

There is no protection.

Problem 2: Too Many Variables

Imagine managing:

- 10 Accounts
- 100 Accounts
- 10,000 Accounts

The number of variables becomes huge.

The code becomes difficult to maintain.

Problem 3: Hard to Manage Large Systems

A bank contains:

- Customers
- Accounts
- Loans
- Transactions
- Employees

Managing all of these using separate variables and functions becomes confusing.

School Analogy

Imagine a school where:

- Student information is stored in one room
- Grades are stored in another room
- Attendance is stored in another room

Every time you need student information, you must visit multiple rooms.

That is inefficient.

Key Message for Students

Procedural Programming works well for small programs, but becomes difficult to manage as software grows larger.

Part 3 — How OOP Was Born

Why Was OOP Created?

They wanted software to behave more like the real world.

Real-World Example: University System

Think about a university.

There are:

- Students
- Teachers
- Courses
- Departments

Humans naturally think about these as separate things.

We do not think:

Step 1
Step 2
Step 3
Step 4

Instead we think:

Student
Teacher
Course
Department

Each thing has its own information and responsibilities.

OOP's Big Idea

The real world is made of objects.

Software should also be made of objects.

Example: Student

Properties

- Name
- Age
- GPA

Actions

- Study
 - Submit Assignment
 - Check Grade
-

Example: Car

Properties

- Color
- Brand
- Speed

Actions

- Start
- Stop
- Accelerate

These become Objects in OOP.

Key Message for Students

OOP was created to help programmers organize large software systems in a way that matches how humans naturally understand the world.

Part 4 — The Four Pillars of OOP

The four pillars are the foundation of Object-Oriented Programming.

1. Encapsulation

Meaning

Keep data and actions together and protect the data from invalid changes.

Real-World Example: ATM Machine

When using an ATM, you can:

- Withdraw Money
- Deposit Money
- Check Balance

But you cannot see:

- The database
- Security system
- Internal cash storage

The internal details are hidden.

Student Summary

Encapsulation means protecting data and allowing only safe access to it.

2. Inheritance

Meaning

A child class can reuse the features of a parent class.

Real-World Example: Person → Student

Every Student is also a Person.

Person Has

- Name
- Age
- Walk()
- Talk()

Student Adds

- Student ID
- Study()
- Submit Assignment()

The Student automatically inherits everything from Person.

Student Summary

Inheritance allows a child class to reuse properties and behaviors from a parent class.

3. Polymorphism

Meaning

The same action can behave differently depending on the object.

Real-World Example: Animal Speaking

Every animal can perform:

```
 speak()
```

But the result is different.

Dog

```
Woof
```

Cat

```
Meow
```

Cow

```
Moo
```

Same method.

Different behavior.

Student Summary

Polymorphism means one action can have many forms.

4. Abstraction

Meaning

Hide unnecessary complexity and show only what users need.

Real-World Example: Driving a Car

You use:

- Steering Wheel
- Brake
- Accelerator

You do not need to understand:

- Engine Timing
- Fuel Injection
- Transmission Logic

The complicated parts are hidden.

Student Summary

Abstraction hides complexity and exposes only what is necessary.

Part 5 — First Java Class

What is a Class?

A **Class** is a blueprint.

It describes what an object should look like.

Real-World Example: House Blueprint

A blueprint defines:

- Number of rooms
- Number of doors
- Number of windows

But nobody can live inside a blueprint.

It is only a design.

What is an Object?

An **Object** is a real thing created from a blueprint.

Example

Blueprint:

Student

Objects:

```
Student s1 = new Student();  
Student s2 = new Student();  
Student s3 = new Student();
```

House Example

Blueprint:

House Design

Objects:

```
House #1  
House #2  
House #3
```

All houses follow the same design but are separate houses.

Student Object Example

```
Student s1 = new Student();  
Student s2 = new Student();
```

If:

```
s1.name = "Sokha";
```

and

```
s2.name = "Dara";
```

Changing Sokha's name does not affect Dara.

Because they are different objects.

Student Summary

A Class is a blueprint. An Object is a real instance created from that blueprint.

Final Recap

What Students Must Remember

Programming Paradigm

A way of organizing and thinking about code.

Procedural Programming

Data and functions are separate.

OOP

Objects combine data and behavior together.

The Four Pillars

Pillar	Easy Meaning	Real Example
Encapsulation	Protect data	ATM Machine
Inheritance	Child reuses parent features	Student → Person
Polymorphism	Same action, different behavior	Dog/Cat/Cow speak()
Abstraction	Hide complexity	Driving a Car

Class vs Object

Class	Object
Blueprint	Real Thing
Student Class	Sokha Student
House Design	Actual House
Car Design	Actual Car

One-Sentence Definition of OOP

Object-Oriented Programming (OOP) is a programming approach that organizes software into objects that contain both data (attributes) and actions (methods), making programs easier to understand, maintain, and expand.